SENTRION

Technical Documentation

20 October 2026

INTERNATIONAL TRACK | “AI as a Financial Participant” | Topic E: SME Finance Copilot

Project	**Sentrion - SME Invoice Operations & Financial Risk MVP**
Application version	0.2.6 (from app/config.py)
Source archive	Sentrion_MVP(6).zip
Source archive SHA-256	52318d8fda664f6d2c447cf8c541553afa748c05205468fe4b09ea9cfac89cdb
Operating model	Local FastAPI application, SQLite database, private local file storage
External AI	Optional; disabled by default
Verification	71 automated tests passed in 9.15 seconds; Python compilation passed

Document purpose and evidence boundary

This document addresses Submission Checklist Item 1: system architecture, core algorithms, and security design. It is based on the inspected source archive, README, configuration, schema, and tests. It describes a local prototype, not a claim of production readiness or regulatory certification. Live Gemini behavior, a hosted deployment, and dataset-wide accuracy for the separate 100-pair dataset were not verified in this documentation run.

Table of Contents

1. Executive Technical Overview....3
2. System Architecture....3
3. Technology Stack and Runtime Environment....5
4. Core Algorithm I - Document Intake and Extraction....5
5. Core Algorithm II - Optional LLM-Assisted Extraction....8
6. Core Algorithm III - Vendor Identity Resolution and PO Relinking....9
7. Core Algorithm IV - Financial Validation and Risk Assessment....10
8. Human Authorization and Workflow Control....11
9. Security Design....12
10. Database and Data Integrity....13
11. Auditability and Evidence Integrity....14
12. Testing and Verification....14
13. Deployment and Reproducibility....15
14. Limitations and Residual Risks....15
15. Conclusion....16
Appendix A. Source-to-Documentation Traceability....16
Appendix B. Verification Record....17

1. Executive Technical Overview

Sentrion is a student-built prototype for small-business finance workflows, with an initial focus on F&B ingredient distributors. It brings supplier invoice intake, purchase-order (PO) evidence, vendor identity resolution, financial consistency checks, risk findings, and human review into one application. The intended outcome is to help finance personnel identify incomplete or inconsistent evidence before they approve a transaction.

Sentrion separates extraction from authorization. Native PDF text extraction, Tesseract OCR, deterministic parsing, and an optional large language model (LLM) help create candidate structured fields. The risk engine then evaluates the stored fields and related records using deterministic rules. Users can inspect findings, correct extracted data, confirm a PO relationship, and follow approval workflows subject to role checks. Neither OCR nor an LLM has authority to approve an invoice or transfer funds.

The current build is a local prototype. It uses FastAPI, Jinja2 templates, SQLite, and private files under a configured local data directory. Payments are simulated records only; no bank connection or real transfer interface is implemented. The application should therefore be evaluated as a controlled sandbox, not as a deployed financial service.

The workflow is designed around three principles:

1. **Evidence before trust.** A supplier name printed on a document is not equivalent to a verified vendor-master record. Automatic PO-to-vendor linking is limited to a unique exact normalized tax-ID match against an eligible verified vendor; name-only candidates require authorized review.
2. **Uncertainty remains visible.** Missing, ambiguous, conflicting, or model-proposed values can require review. A processing job completing does not mean extraction is complete or financially correct.
3. **Risk and authorization remain separate.** GREEN/YELLOW/RED are decision-support classifications. A GREEN invoice still passes server-side approval checks, and unrelated findings remain active after a specific identity issue is resolved.

2. System Architecture

2.1 Architectural overview

The system is a single FastAPI application serving HTML pages and handling form-based workflows. The browser communicates with routes in `app/main.py`. Those routes use authentication/session helpers, the database context manager, extraction and workflow services, risk assessment, audit functions, and the Excel exporter.

Document intake is performed locally. For invoices, the upload route stores the document and enqueues `process_invoice_batch` through FastAPI `BackgroundTasks`. PO extraction is performed in the PO upload route before the response is completed. The extraction pipeline can call an external OpenAI-compatible Chat Completions endpoint if external AI is enabled and an API key is configured; otherwise, it retains local extraction results.

The current build does not use Supabase or Vercel as its validated persistence architecture. Although Sentrion has previously been discussed in the context of a Vercel deployment, the exact archive documented here is explicitly described by its README as a local SQLite prototype, and this archive's hosted deployment has not been verified.

Figure 1. Sentrion local prototype architecture

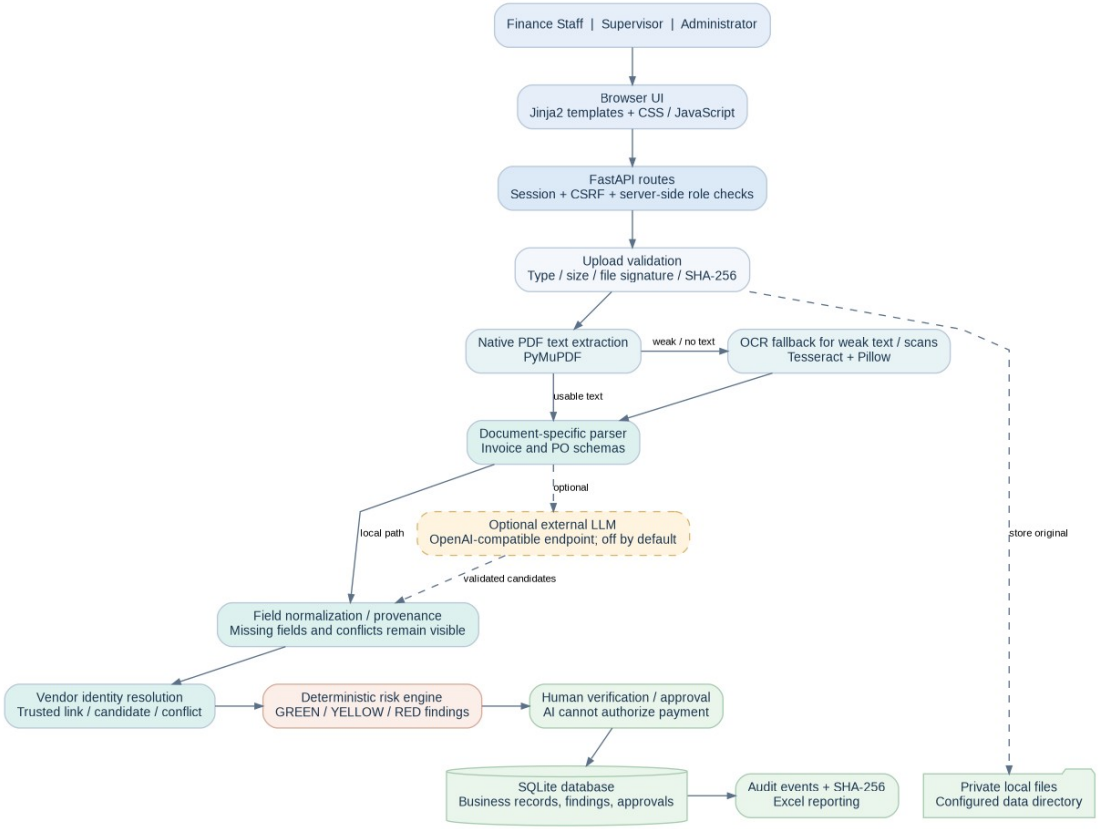


Figure 1. Sentrion local prototype architecture. The external LLM is an optional path; human users remain responsible for review and authorization.

2.2 Component responsibilities

Component	Responsibility	Main implementation
Web interface	Dashboard, invoice and PO lists/details, vendor/approval pages, audit view, forms	app/templates/, app/static/, app/ui_text.py
Application routes	Authentication entry points, authorization checks, uploads, corrections, matching, review, approval, downloads, exports	app/main.py
Configuration	Application name/version, environment loading, data paths, session settings, upload limits, optional AI toggle and currency default	app/config.py
Authentication/security helpers	Password hashing/verification, current-user lookup, CSRF token helpers, role checks, account masking	app/security.py
Database layer	SQLite schema, foreign keys, connection lifecycle, initialization and forward-compatible column additions	app/db.py
Extraction	Native PDF text, OCR fallback, invoice/PO parsing, field normalization, heuristic AI-directed instruction detection	app/services/extraction.py, app/currency.py
Optional LLM adapter	OpenAI-compatible request, JSON validation, bounded retries/time budget, fill-missing merge and safe trace metadata	app/services/llm_extraction.py
Invoice processing	Processing state, field persistence, line items, extraction trace and risk evaluation	app/services/processing.py
Vendor resolution	Tax-ID/name normalization, match classification, trusted link, PO reconciliation/relink and reassessment	app/services/vendor_resolution.py
Risk engine	Creates current findings and calculates maximum active severity	app/services/risk.py
Audit service	Persists timestamped event type/result, actor, reason, details and related document hash	app/services/audit.py
Reporting	Workbook with Invoices, Invoice Items, and Audit Events sheets	app/services/exporter.py
Local storage	SQLite database and uploaded source/evidence files	SENTRION_DATA_DIR, SENTRION_DB_PATH and app/config.py

2.3 Request and persistence boundaries

- FastAPI routes are the principal request boundary. Most state-changing forms validate a CSRF token and apply server-side role checks before a database mutation.
- get_db() opens a SQLite connection with row objects, enables foreign-key enforcement and a busy timeout, commits on successful exit, rolls back on exceptions, and closes the connection.
- The upload layer stores original invoice/PO files beneath the configured upload directory using a generated filename and computes a SHA-256 digest.
- The database persists extracted fields, line items, status/provenance, risk findings, approval records, simulated payments and audit events. The raw extracted text for invoices and PO documents is also retained in the local database/source record model.
- Private document handlers require an authenticated user and constrain resolved paths to the configured upload directory. They return private/no-store response headers for PO document downloads.

3. Technology Stack and Runtime Environment

The source archive identifies application version 0.2.6. The archive has no Git metadata, so no commit hash can be attached to this documentation.

Technology	Evidence in source	Use
Python 3.11+	README requirement; Python package/runtime	Application and tests
FastAPI (>=0.115,<1.0)	requirements.txt, app/main.py	HTTP routes and application lifecycle
Uvicorn (>=0.30,<1.0)	requirements.txt, README launch command	Local ASGI server
Jinja2 (>=3.1,<4.0)	requirements.txt, templates	Server-rendered HTML
SQLite	app/db.py	Local persistence and relational records
PyMuPDF / fitz (>=1.24,<2.0)	requirements.txt, extract_document()	PDF text extraction and rendering scanned PDF pages for OCR
Tesseract via pytesseract (>=0.3.10,<1.0)	requirements.txt, _ocr_pil_image()	OCR for scan/image fallback
Pillow (>=10,<13)	requirements.txt	Image normalization and validation
python-multipart	requirements.txt	FastAPI form/file uploads
itsdangerous	requirements.txt; Starlette session middleware integration	Session signing dependency
openpyxl (>=3.1,<4.0)	requirements.txt, services/exporter.py	Excel workbook generation
pytest (>=8,<9) and HTTPX (>=0.27,<1.0)	requirements-dev.txt	Regression and web-workflow tests

The one-click Windows script is SETUP_AND_RUN.bat. The README states that it checks Python 3.11+, creates or reuses a shared virtual environment under %LOCALAPPDATA%\Sentrion, installs requirements when needed, checks Tesseract, creates .env if missing, launches the server and opens the browser. It preserves an existing .env and dependency cache.

4. Core Algorithm I - Document Intake and Extraction

4.1 Intake and source-file validation

Invoice and PO routes accept PDF, PNG and JPEG. check_upload_bytes() checks extension, file size (configured by SENTRION_MAX_UPLOAD_MB, default 10 MB), PDF magic bytes or image readability. The upload is stored under a random UUID-based filename; SHA-256 is computed over the original bytes. Uploaded display filenames are treated as metadata rather than trusted storage paths.

Vendor onboarding evidence uses a separate validation path that supports additional formats (including Office and text files), checks signatures/ZIP structure where applicable, caps files per evidence category, calculates SHA-256, and verifies attachment metadata and bytes when the evidence is reviewed. This broader evidence-upload format list does not mean those formats are accepted as invoice/PO documents.

4.2 Native text and OCR fallback

For PDF pages, extract_document() obtains text via PyMuPDF. A page is considered usable only when it meets several heuristics: at least 80 characters, at least eight alphanumeric words of length two or more, and at least 50 alphanumeric characters. If text is not usable, the page is rendered at 2.5 scale and OCR is attempted. OCR normalizes an image with grayscale, autocontrast, optional upscaling toward 1600 pixels on the longest dimension, and contrast enhancement. Tesseract is called with Indonesian and English language data (ind+eng) using page segmentation mode 6; an English-only fallback is attempted if that fails.

For PNG/JPEG, the image is EXIF-transposed, converted to RGB, normalized, and OCRed. If the resulting text is empty, extraction fails visibly rather than returning an apparently complete empty record. The OCR binary and language packs are external system prerequisites; an application startup alone does not prove OCR is available.

4.3 Structured field parsing

extract_fields() builds document-specific structured output for invoices and purchase orders. The parser uses label variants, section/party context, header identifiers, dates, amounts, currency evidence and table-like line-item layouts. It tries to distinguish supplier from buyer based on role labels and document type. It normalizes dates only where the date order is sufficiently unambiguous; ambiguous numeric dates are not guessed.

The output can include document identifiers, supplier/buyer identity and tax IDs, issue/due dates, PO reference, bank details, currency, subtotal/discount/tax/shipping/rounding/grand total and line items (SKU, description, quantity, unit, unit price, discount, tax and line total). Not every document supplies every field. Missing values are intended to remain missing rather than be manufactured.

currency.detect_currency() treats explicit currency codes and recognized currency evidence as stronger than a fallback. An unqualified $ is resolved using the configured default (USD by default) and marked POLICY; no reliable currency evidence returns a LOW confidence grade. The policy is application-specific, not a claim that $ universally means USD. parse_money_amount() uses decimal-aware parsing and currency-sensitive display formatting; it does not perform exchange-rate conversion.

4.4 Completeness state versus processing state

The invoice processor marks processing_status as COMPLETED when the processing workflow finishes. Separately, it marks extraction_status as PARTIAL when any of four required fields is missing: invoice number, vendor name, invoice total or due date. Thus, a completed job can still require review. Risk evaluation adds a YELLOW EXTRACTION_INCOMPLETE finding for a partial or unsuccessful extraction.

PO records similarly persist total_status and total_source. If a reliable PO grand total is missing, the numeric total column may receive 0 for schema compatibility, but the status remains MISSING/UNKNOWN; risk assessment must treat the total as unavailable rather than compare that placeholder as a real amount. An external-LLM proposed total is marked REVIEW_REQUIRED until independently confirmed.

Figure 2. Document intake, extraction, validation, and review

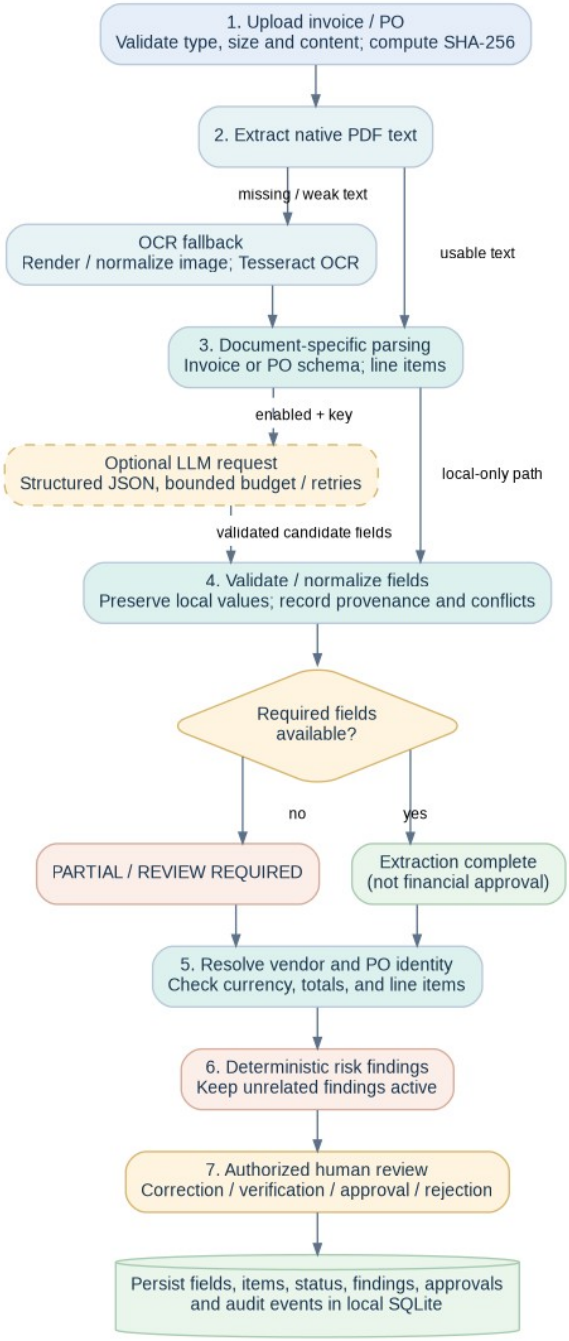


Figure 2. Document intake and processing path, including optional LLM-assisted extraction and the human review boundary.

Extraction pseudocode

```
accept upload
validate supported file type, size, and file content
compute SHA-256 and store original in private local storage
if PDF:
   extract text from each page
   if a page's text is insufficient:
      render page and attempt Tesseract OCR
else if image:
   normalize image and run Tesseract OCR
parse document using the invoice or PO schema
if optional external AI is enabled and configured:
   request structured JSON within bounded time/retry limits
   validate result and fill missing local fields only
   preserve local non-empty values and record disagreements
resolve currency from source evidence/configured policy
normalize dates, monetary amounts, identifiers and line items
persist fields, provenance, source hash and processing status
run deterministic risk assessment and preserve audit history
route unresolved fields/findings to human review
```

5. Core Algorithm II - Optional LLM-Assisted Extraction

5.1 Activation and protocol

External AI is disabled by default. `EXTERNAL_AI_CONFIGURED` requires both `SENTRION_ENABLE_EXTERNAL_AI=true` and a non-empty `OPENAI_API_KEY`. The `.env.example` configures an OpenAI-compatible base URL and model; it notes the compatible Gemini endpoint `https://generativelanguage.googleapis.com/v1beta/openai`. The adapter builds a `/chat/completions` request using bearer authorization, temperature 0, JSON-object response formatting, and separate document-type wording for supplier invoices and purchase orders.

The provider is classified from the base URL for trace metadata (for example, Google Gemini, OpenAI, or a generic OpenAI-compatible provider). Provider classification is metadata; it does not prove the endpoint/model combination has been tested against a live service.

5.2 Timeouts, retries, and fallback

Defaults in `.env.example` are 12 seconds per request, a 24-second total time budget, and one retry. The code bounds these settings (request timeout at 1-60 seconds, total budget at least the per-request timeout and no more than 90 seconds, retries from 0 to 3). Retryable HTTP response codes include 408, 425, 429, 500, 502, 503 and 504; timeouts may also be retried. A small bounded backoff is used. Authentication/invalid-request failures are not retried indefinitely.

The adapter returns local extraction results when AI is disabled or the key is missing. On a request, response, or validation failure it records a sanitized error type and uses local results as fallback. It does not log the API key, response body, request headers or raw document text in the extraction trace.

5.3 Structured-output validation and merge policy

The adapter only accepts known fields, validates that the response is a JSON object, checks scalar/list shapes, and filters malformed line items. It fills missing local scalar values and can fill an empty local item table. If a model proposes a different non-empty value, the local value is retained and the conflict is recorded for review. Currency is independently detected from source text and policy; it is not trusted solely from model output.

The prompt treats document text as untrusted evidence and instructs the model not to follow embedded instructions to change policy, approve payments or reveal data. This reduces risk but is not a complete prompt-injection defense. The risk engine separately applies a heuristic pattern detector for suspicious instructions. Model output is still potentially incorrect and requires deterministic validation and human review where appropriate.

LLM requests include up to 45,000 characters of extracted document text. If enabled, this means document content is sent to the configured external provider. Only synthetic or expressly authorized data should be used unless data-handling arrangements permit external processing.

No live Gemini/provider request was performed for this documentation validation. The 71-test suite includes mocked tests for provider success, disabled/missing-key handling, malformed JSON, retryable HTTP errors, PO-specific extraction and trace safety. These tests verify adapter behavior under controlled responses, not real provider uptime or extraction accuracy.

6. Core Algorithm III - Vendor Identity Resolution and PO Relinking

6.1 Normalization and matching policy

vendor_resolution.normalize_tax_id() applies Unicode NFKC normalization, case folding and removal of non-alphanumeric characters while preserving meaningful letters and leading zeros. normalize_legal_name() applies Unicode NFKD, removes combining marks, case-folds, and retains normalized alphanumeric tokens. The name matcher is exact after normalization; it is not a fuzzy or substring matcher.

The resolver distinguishes the supplier identity extracted from the PO from the vendor_id relationship to the trusted vendor master. The primary automatic match is a unique exact normalized tax-ID match to a vendor whose verification_status is VERIFIED. A same-name candidate without a reliable exact tax-ID match remains a candidate for authorized review. Multiple candidates or conflicting reliable tax IDs are not automatically trusted links.

A manual link requires a verified target vendor, a reason, and the matching evidence rules. A name-only confirmation requires explicit reviewer confirmation and a reason; a conflicting tax-ID override requires supervisor/admin control and a longer review note, and the conflict remains a risk finding. An existing link to another vendor cannot simply be overwritten.

6.2 Relinking previously uploaded purchase orders

relink_unlinked_purchase_orders() inspects unresolved POs after a vendor has become verified. It automatically links only exact normalized tax-ID matches. Exact name matches without that exact tax-ID evidence are recorded as candidates; reliable tax-ID conflicts are marked as identity conflicts. The logic is designed to be idempotent: if the PO is already linked to the same vendor, it does not create another relationship.

link_po_to_vendor() persists the link method, reason, time and actor, writes a PO_VENDOR_LINKED audit event, and calls _reassess_related_invoices() for invoices whose PO number matches. Reassessment runs evaluate_invoice(), which supersedes the prior active findings and creates a new active analysis run. The historical vendor-link finding remains in the finding history as SUPERSEDED; unrelated active findings are recomputed and remain active if still applicable.

The vendor onboarding final-approval route calls relinking after the new vendor and its verified payment destination are created. The purchase-order detail route also provides a Supervisor/Admin-only reconciliation action. These routes do not make an invoice payment-ready merely by establishing a vendor relationship.

Figure 3. Vendor identity resolution and PO relinking

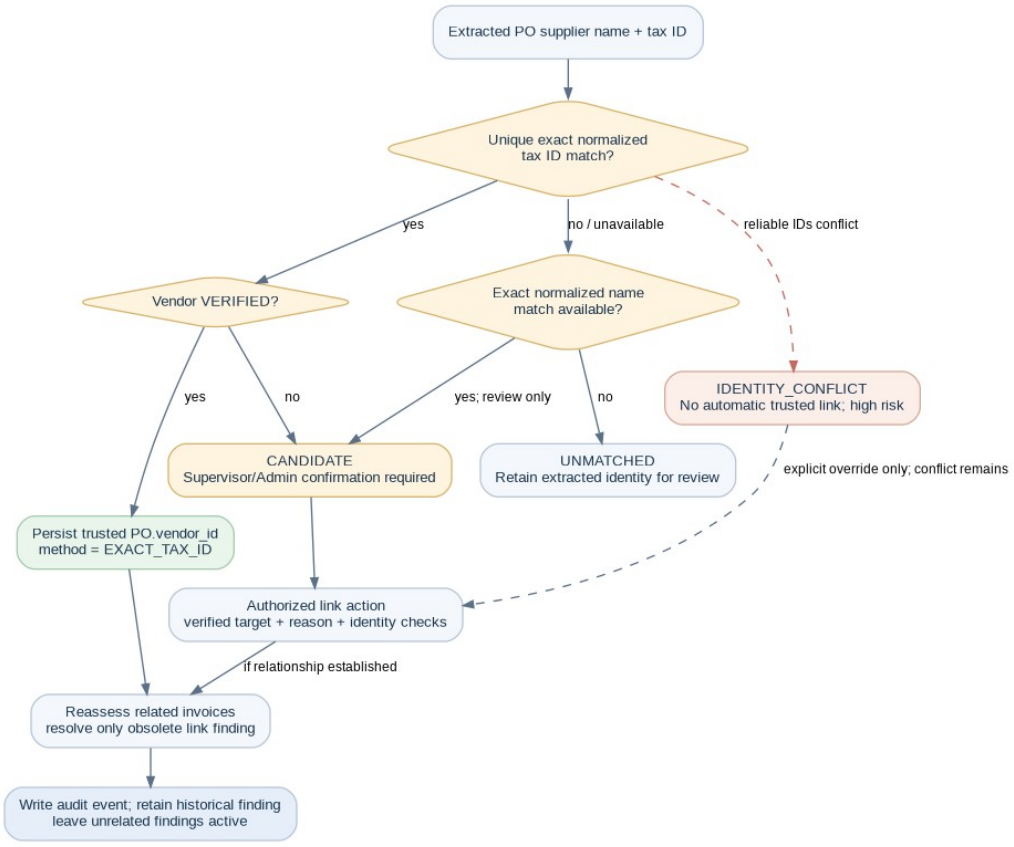


Figure 3. Deterministic identity resolution, controlled relinking, audit logging and risk reassessment.

Vendor-resolution pseudocode

```
load PO and candidate vendor
if PO already linked to this vendor:
   return ALREADY_MATCHED
require target vendor verification_status == VERIFIED
if PO tax ID and vendor tax ID are both reliable and exactly match after normalization:
   persist trusted link using EXACT_TAX_ID
elif reliable tax IDs conflict:
   reject automatic link; require explicit authorized conflict review
elif normalized legal names match exactly:
   require authorized name-only confirmation and review reason
else:
   leave PO unmatched or as a candidate
if a link is established:
   record audit event
   reassess invoices referencing the PO number
   retain previous findings as SUPERSEDED history
```

7. Core Algorithm IV - Financial Validation and Risk Assessment

7.1 Findings model

`evaluate_invoice(conn, invoice_id)` loads the invoice and its persisted metadata, gathers applicable findings, computes a severity level, marks current active findings from the previous analysis as `SUPERSEDED`, writes new `ACTIVE` findings under a new `analysis_run_id`, and updates the invoice's `risk_level` and reason list.

Severity ordering is `GREEN < YELLOW < RED`; the result is the highest severity among current findings. GREEN means no finding reaches the configured escalation level and required evidence is adequate according to the rules. YELLOW means review or verification remains necessary. RED denotes a high-risk condition that blocks the standard approval route. These are prototype rules, not proof of fraud, a universal accounting policy, or certification.

7.2 Principal checks implemented

Check	Behavior in current implementation
Currency confidence	LOW-confidence invoice currency produces YELLOW CURRENCY_UNVERIFIED. Currency mismatch with the matched PO produces RED and suppresses monetary comparison across currencies. No exchange conversion is performed.
AI-directed instructions	Heuristic pattern matches in invoice text produce RED AI_DIRECTED_INSTRUCTION; the document content is treated as data, not instructions. Detection is not exhaustive.
Vendor resolution	Unverified/missing/ambiguous vendor generally produces YELLOW. Supplier name or tax-ID mismatch and reliable tax-ID conflict produce RED.
Payment destination	An invoice account not present in the verified vendor bank-account set produces YELLOW BANK_ACCOUNT_CHANGED; a holder mismatch against verified bank history can be RED. An inferred company-name holder fallback is not treated as independent ownership evidence. Missing bank account produces YELLOW.
Required fields	Missing invoice number, supplier name, total or due date produces YELLOW MISSING_REQUIRED_FIELD. PARTIAL/incomplete extraction adds YELLOW EXTRACTION_INCOMPLETE.
Subtotal arithmetic	If all printed line totals exist, their sum is used when it is within tolerance of printed subtotal; otherwise, complete discounts allow net item base, sum(quantity * unit_price - line_discount). If discounts are unavailable but quantities/prices exist, gross sum sum(quantity * unit_price) is used. A material difference produces YELLOW SUBTOTAL_MISMATCH.
Duplicate invoice	Duplicate invoice number for the same vendor creates RED DUPLICATE_INVOICE.
PO reference	Missing PO reference or unavailable PO number produces YELLOW. PO identity conflict or a different linked vendor produces RED. A matching supplier name without a persisted trusted PO link produces YELLOW PO_VENDOR_LINK_REVIEW.

Check	Behavior in current implementation
PO grand total	MISSING/UNKNOWN total status produces YELLOW PO_TOTAL_UNAVAILABLE; an external-model candidate with REVIEW_REQUIRED produces YELLOW PO_TOTAL_REVIEW_REQUIRED. Placeholder zero is not treated as an extracted grand total.
Invoice/PO total variance	With currency evidence compatible and reliable totals available, a difference above 10% of PO total is RED for most currencies; for IDR, RED threshold is the greater of 10% of PO total or IDR 500,000. A smaller difference exceeding rounding tolerance produces YELLOW PO_AMOUNT_VARIANCE. These are demo thresholds.
Line-item comparison	Quantity difference above 0.001 produces YELLOW. Unit-price difference above a 5% tolerance (with currency-specific minimum tolerance) produces YELLOW. PO items absent from the invoice produce YELLOW. Comparison requires compatible currencies and available fields.
Historical amount anomaly	If at least three prior comparable invoices are available, a total above 3 times or below 30% of the historical median produces YELLOW AMOUNT_ANOMALY.

For subtotal arithmetic, tolerance is $\max(100, 2\% \text{ of subtotal})$ for IDR and $\max(0.01, 2\% \text{ of subtotal})$ for other currencies. Price variance uses a 5% tolerance with a minimum of IDR 1,000 or 0.01 for other currencies. These are implementation-specific demo settings and require validation against real business rules.

7.3 Approval gates

The standard invoice approval route reevaluates current facts at the time of approval. It blocks approval if risk is not GREEN, processing/extraction is incomplete, the invoice was rejected, an invoice/vendor/total field is invalid, the bank account is not in verified vendor bank records, or a simulated payment already exists. The route then writes an idempotent simulated payment record only; it does not send funds to a bank.

A two-person RED-resolution request exists for eligible PO mismatch exceptions. It binds the exception to reviewed invoice total, PO total and currencies; it does not generally clear other RED findings or bypass the normal approval path. The final invoice approval still calls the risk engine and must satisfy the standard GREEN/bank-account checks.

8. Human Authorization and Workflow Control

Role	Verified responsibilities and restrictions
Finance (finance)	Can access invoice/PO and vendor workflows; upload invoices and POs; submit vendor onboarding or bank/profile change requests; correct/verify invoices; request eligible exception review; approve invoices only through the server-validated GREEN path. Cannot manually confirm PO/vendor identity links and cannot act as final approver where supervisor/admin is required.
Supervisor (supervisor)	Has Finance workflow access and can perform final approval for multi-stage requests; can establish/reconcile PO-to-vendor links with a documented reason; can perform reviewed administrative actions exposed by routes. Must not be the requester or first approver on a request they finalize.
Administrator (admin)	Has the same broad route permissions as Supervisor for documented workflows, including final approval and PO vendor-link operations. The source does not imply access to real banking operations because no such integration exists.

Vendor onboarding and bank-account change requests use maker-checker controls: the requester cannot approve their own request; distinct people complete first and final approval; final approval is restricted to supervisor/admin; evidence files are required for vendor onboarding and are rechecked for integrity; review confirmation and request-payload fingerprints protect the approval from changed-after-review data. `approval_records` includes a uniqueness constraint on `(request_id, user_id)`.

The interface hides or shows actions based on permissions, but server-side route checks are the effective control. CSRF tokens are also checked in state-changing form routes. The specific route logic and tests should be reviewed when changing permissions; UI visibility alone must never be treated as access control.

9. Security Design

9.1 Authentication and session handling

app/security.py hashes passwords using PBKDF2-HMAC-SHA256 with 310,000 iterations and a random 16-byte salt. The stored value records the scheme, iteration count, salt and derived digest. Verification uses constant-time hmac.compare_digest. current_user_from_request() reads the session user ID and checks that the account still exists and is active before allowing the request to proceed.

Starlette SessionMiddleware is configured with cookie name sentrion_session, an eight-hour maximum age, SameSite=Lax, and an https_only flag controlled by SENTRION_HTTPS_ONLY. The local example uses false for localhost. SESSION_SECRET has a development fallback in code; it must be replaced with a strong, private random secret before any shared/hosted environment, and SENTRION_HTTPS_ONLY=true must be used behind HTTPS. The fallback means the application does not fail closed solely because the secret is unset, so configuration review is a deployment prerequisite.

CSRF tokens are generated with secrets.token_urlsafe(32) and compared using hmac.compare_digest. Role checks are performed server-side through require_roles().

9.2 Upload, file and evidence integrity controls

Invoice/PO uploads have a size ceiling, extension checks, PDF magic-byte validation, and image validation through Pillow. Stored filenames are generated UUIDs rather than user-supplied paths. Document SHA-256 hashes are stored with metadata and included in relevant audit events. Private document handlers verify that resolved paths remain under the configured upload directory. Vendor evidence files are stored in a separate directory, hashed, and compared to expected metadata and contents when accessed/reviewed.

These controls reduce path manipulation and detect changes relative to a recorded hash. They do not prove that an invoice is authentic, that a vendor owns an account, or that the SQLite file/audit events are immutable.

9.3 Threat model and residual risk

Threat	Implemented mitigation	Residual risk / limitation
Malformed/unsupported upload	Extension, size, PDF signature and image validation; upload error on invalid format	Content validation is not a full malware scanner or sandbox. PDF and image libraries still process untrusted input.
Supplier identity spoofing	Exact normalized tax-ID match for automatic trusted linking; name-only candidate/reviewer confirmation; conflicting tax IDs remain high-risk	Source-document identity may itself be false; there is no independent government/tax registry verification.
Bank-account substitution	Compare invoice account with verified vendor bank-account records; bank changes require evidence and separate approvals; normal UI/export masks account number	No bank API confirms ownership. A company-name fallback is inferred, not independent evidence.
Duplicate invoice / double payment	Duplicate invoice risk finding; payments.idempotency_key is UNIQUE; standard approval checks for existing simulated payment	Prototype/local database constraints are not a substitute for financial institution settlement controls.
Unauthorized role action	Server-side role guards; CSRF checks; user lookup from session; dual approval restrictions	Every new route still needs permission review and regression tests; local deployment security depends on configuration and host access.
Approval tampering	Review confirmation, payload fingerprint check, distinct requester/approver controls, unique approval record per user/request	Local SQLite records remain mutable to a user with filesystem/database access.
Prompt injection via document	LLM prompt identifies document text as untrusted; regex-based AI-directed instruction detector creates RED finding	Detection is heuristic and can miss or falsely flag instructions; a model can still produce wrong outputs.
Secret leakage	.env configuration, no API key in extraction trace, sanitized LLM error metadata	Secrets can still leak through operator misconfiguration, environment dumps, deployment logs, or repository mistakes. The .env file must not be committed.
Unauthorized source-file access	Authenticated handlers, resolved path constrained to upload directory, private/no-store headers on PO download	This is a local app, not a hardened multi-tenant file service; host-level file permissions/backups require care.
Incorrect extraction / misclassification	Provenance, missing-field states, deterministic risk checks, human review, no automatic bank transfer	OCR/regex/LLM extraction can be wrong; risk thresholds are demo policies; reviewers must inspect material values.
Audit evidence tampering	Timestamped audit rows and document hashes	SQLite audit trail is not cryptographically chained, externally anchored, or immutable. An operator with write access could alter both data and local records.

9.4 LLM boundary

External AI is opt-in. When enabled, extracted document text is sent to the configured provider. The LLM's job is candidate field extraction only. The local application validates shape and merges into missing fields; it does not allow model output to invoke approval routes. The prompt includes instruction isolation, but this is not a guarantee against prompt injection. Live provider reliability and endpoint compatibility were not tested as part of this archive validation.

10. Database and Data Integrity

10.1 Core data model

The current schema is SQLite and is defined in app/db.py. The main entities and relationships are illustrated below.

Figure 4. Core SQLite entity relationships

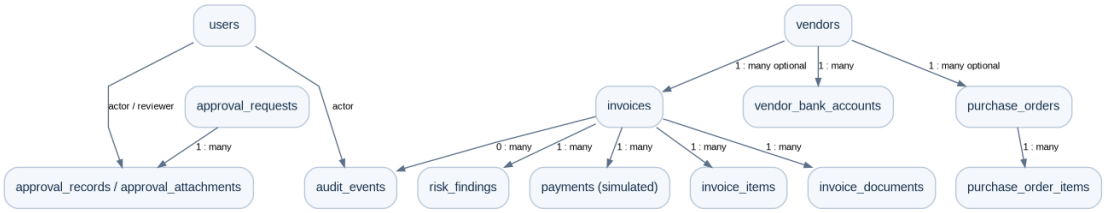


Figure 4. Main schema entities and their principal relations. Invoice-to-PO association is stored as invoices.purchase_order_number text rather than a declared foreign key to purchase_orders.

Entity	Purpose and selected fields
users	User identity, unique email, password hash, role and active flag
vendors	Vendor master, tax ID, verification state and current payment destination
vendor_bank_accounts	Historical/current account records, status, verifier and verification timestamp
purchase_orders	Unique PO number, optional trusted vendor_id, extracted supplier/buyer values, totals, currency, document fields and vendor-match state/method/reason
purchase_order_items	PO item name, SKU, description, quantity, unit, price, discount, tax and line total
invoices	Invoice identity and money fields, vendor_id, PO number text, extraction/processing/verification/approval/payment states, risk level and source metadata
invoice_items	Invoice line items and amounts
invoice_documents	Original filename/path, SHA-256, size, MIME type, uploader and timestamp
risk_findings	Finding code/severity/detail, expected/observed values, source, status and analysis run ID
approval_requests	Generic approval request, entity type/ID, requester, evidence, review context and status timestamps
approval_records	Per-user decisions and reasons; unique (request_id, user_id)
approval_attachments	Evidence attachment metadata and SHA-256
payments	Simulated payment status, reference, amount/currency and unique idempotency key
audit_events	Event type/result, actor, reason/details, related approval and document hash

The database schema declares foreign keys for many one-to-many relationships (for example, vendor-to-bank accounts, invoice-to-items/findings/documents/payments, PO-to-items, approval request-to-records/attachments, and user references). An invoice references a PO by its PO number string rather than a declared foreign key, so risk evaluation explicitly looks up the PO by normalized/case-insensitive PO number. That design makes PO number uniqueness and validation especially important.

10.2 Initialization and migrations

init_db() creates missing tables/indexes and adds selected missing columns to existing SQLite tables. Before particular vendor-management/schema changes, _backup_before_vendor_migration() copies a non-empty database to a timestamped backup file when it detects legacy schema conditions. Forward additions preserve existing local records; migration logic is SQLite-specific and is not a cross-database migration system.

The local database and uploads are stored under SENTRION_DATA_DIR (default ./data); SENTRION_DB_PATH can override the database file path. Those runtime assets must be kept out of public source repositories and submission archives.

11. Auditability and Evidence Integrity

The audit service writes event rows with UUID, optional invoice and actor references, actor role, event type/result, risk level at event time, reason, JSON details, related approval, document hash and timestamp. Events are used to explain extraction provider selection/fallback, uploads, vendor links/relinks, approval actions, risk reassessment and simulated payment operations where the relevant workflow emits events.

Extraction trace records are deliberately summarized. They may include document type, provider label/host, model identifier, attempted/status flags, whether fallback was used, error type, recognized field names, conflict field names, retry count, latency and processing-run ID. They do not include the API key or raw document body. Some route-specific audit details include business context; sensitive identifiers should continue to be omitted or masked in new events.

SHA-256 establishes a digest for the bytes that were hashed, allowing later comparisons against a stored digest. It does not establish the source's identity, content truth, account ownership or legal authenticity. The local SQLite audit log is neither append-only at the database layer nor tamper-proof. A production-grade audit model would need independently protected logs, backups, access governance and retention procedures.

Excel export produces three worksheets: Invoices, Invoice Items, and Audit Events. Account numbers are masked in invoice export rows, and strings that resemble formulas are explicitly kept as text to reduce spreadsheet-formula injection risk.

12. Testing and Verification

12.1 Commands run against the exact supplied archive

The latest supplied archive was unpacked into a separate working directory. No application code or business data was modified for this documentation run.

```
python -m pytest -q
..................................................................... [100%]
71 passed in 9.15s

python -m compileall -q app tools_*.py tests
COMPILEALL_PASS
```

The tests run successfully in this environment. No Git commit was available to identify the archive by commit hash.

12.2 Coverage present in the repository

Test module	Areas covered by named tests
tests/test_currency.py	Currency formatting/detection, decimal amount parsing, currency mismatch and confirmation behavior
tests/test_db_migrations.py	Vendor-management and PO vendor-link schema migrations with preservation of existing data/relationships
tests/test_extraction_risk.py	Local extraction, scanned-image OCR, prompt-injection detection, risk example levels, field variants and ambiguous dates
tests/test_llm_traceability.py	Mock provider success/failure, disabled/missing-key behavior, malformed JSON fallback, PO-specific prompt, bounded retry and secret-free trace
tests/test_revision_features.py	Dataset-v4 extraction cases, dollar policy, PO upload/detail, missing grand total behavior, account-holder inference and LLM total review
tests/test_vendor_resolution.py	Exact tax-ID matching, name-only review, tax-ID conflict, ambiguity, PO relinking/idempotency, risk finding history and unauthorized link denial
tests/test_web_workflows.py	Login/pages, uploads, mock approval/payment controls, exports, dual authorization, onboarding/evidence integrity, filters and incomplete extraction consistency

12.3 What the passing suite does and does not establish

The 71 passing tests establish the behavior asserted by these tests and fixtures. The current archive includes a synthetic example set and test fixture documents, but it does not include the full 100-pair Dataset v4 archive mentioned in earlier project work. Therefore, no full 100-pair extraction benchmark or 100-pair end-to-end transaction benchmark was performed for this documentation deliverable.

External LLM tests use mocked responses. No real Gemini/OpenAI-compatible provider request, external API reliability test, public internet deployment smoke test, or hosted production test was performed. The current README explicitly identifies this archive as a local SQLite prototype and says that no verified hosted deployment is included. Passing repository tests is not a substitute for those separate validations.

13. Deployment and Reproducibility

13.1 Local setup

Prerequisites are Python 3.11+ and Tesseract OCR with language data accessible on PATH for scanned documents. First-time installation requires access to the Python package index. External AI is not required for local-only mode.

Windows one-click: extract the full application folder to a writable location and run SETUP_AND_RUN.bat. Follow the README instructions; keep the server console open, use http://127.0.0.1:8000, and stop with Ctrl+C.

Manual Windows setup: from the directory containing app/main.py:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
# Edit .env; set a strong private SENTRION_SESSION_SECRET.
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Manual macOS/Linux setup:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
cp .env.example .env
# Edit .env; set a strong private SENTRION_SESSION_SECRET.
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

13.2 Relevant configuration

Variable	Purpose / default documented in .env.example
SENTRION_SESSION_SECRET	Private signing secret; replace placeholder before shared use
SENTRION_HTTPS_ONLY	Cookie Secure setting; false is for localhost only
SENTRION_MAX_UPLOAD_MB	Per-file upload limit; default 10
SENTRION_DEFAULT_CURRENCY	Configured fallback currency; default USD
SENTRION_DATA_DIR	Local data/upload directory; default ./data
SENTRION_DB_PATH	SQLite database path; default ./data/sentrion.db
SENTRION_ENABLE_EXTERNAL_AI	External structured extraction toggle; default false
OPENAI_API_KEY	API key for the configured compatible provider; default empty
OPENAI_BASE_URL	OpenAI-compatible endpoint base; default OpenAI API URL
OPENAI_MODEL	Model name; default in .env.example is gpt-4.1-mini
SENTRION_AI_TIMEOUT_SECONDS	Per-request timeout; default 12
SENTRION_AI_TOTAL_BUDGET_SECONDS	Total request budget; default 24
SENTRION_AI_MAX_RETRIES	Bounded retry count; default 1

Local demo accounts are documented in README for local testing only. Do not expose or reuse those credentials on a public/shared system. The source package's seed mechanism calls seed_demo_data() during application startup, so a fresh local sandbox receives its seeded demo records.

13.3 Hosted deployment status and limitations

The README describes a private persistent-host deployment as guidance, not a verified configuration. It explicitly warns against deploying this SQLite/local-filesystem build to a serverless platform such as Vercel without a separately reviewed persistence/storage design and platform-specific migration. Consequently, this technical document does not claim that the exact archive has been deployed or smoke-tested in production.

14. Limitations and Residual Risks

1. **Prototype risk settings.** Thresholds are demonstration rules and should not be treated as universal accounting policy.
2. **OCR and parser variance.** New layouts, low-quality scans, handwritten content, unusual separators or tables can cause missed or incorrect fields. Human confirmation remains necessary for material values.

3. **External AI variability.** Provider availability, latency, endpoint compatibility, rate limits and extraction quality vary. Mock tests do not verify live Gemini reliability.
4. **Prompt-injection detection.** The regex-based detector and model prompt are defense-in-depth only; they cannot guarantee detection of every malicious document instruction.
5. **Vendor/bank verification.** Extracted company name and inferred account holder are not independent verification. There is no direct bank or tax-registry integration.
6. **Financial matching scope.** The README says goods-receipt matching is not implemented. PO/invoice matching does not establish that goods were delivered or accepted.
7. **Payments.** The app records simulated payments only; it does not perform settlement or connect to a bank.
8. **Persistence.** SQLite plus local file storage is suitable for a controlled local demonstration but requires a separate architecture for shared, horizontally scaled or serverless hosting.
9. **Audit immutability.** The audit table and database file are not immutable or cryptographically chained.
10. **Submission artifacts.** The source README links to `SECURITY_SELF_ASSESSMENT.md`, but that separate file was not present in the supplied `Sentrion_MVP(6).zip`. This document contains a security design and threat-model section, but the separately named self-assessment should be added to the submission repository if the checklist expects it as a distinct artifact.
11. **Repository identity.** The README still contains a GitHub repository URL placeholder. The final submission must replace it and verify the public commit matches the exact source ZIP.
12. **Third-party licenses.** The archive does not declare a separate Sentrion source-code license. Dependency and asset licenses should be reviewed before public distribution; do not assume the project is released under an open-source license.

15. Conclusion

Sentrion combines local document extraction, optional LLM-assisted field proposals, deterministic vendor resolution, invoice/PO financial checks, risk findings, role-based review and simulated payment recording in one FastAPI prototype. Its key safety boundary is that extracted data remains evidence to validate, not an authorization instruction. Trusted vendor links require stronger identity evidence or an authorized review; risk findings are recalculated as relationships change; and approval/payment routes reevaluate current conditions before recording a simulated payment.

The inspected version (`0.2.6`) is reproducible locally and the repository's 71 automated tests and Python compilation checks passed in this validation run. It remains a prototype with local SQLite/file persistence, configurable demonstration thresholds, OCR/provider limitations, no real bank integration and no verified hosted deployment for this exact archive.

Appendix A. Source-to-Documentation Traceability

Documentation claim	Source reference	Verification method	Status
Application version and local configuration	app/config.py: APP_VERSION, paths, session, upload, AI and currency settings	Source inspection	Implemented; configuration values depend on environment
FastAPI lifecycle and session setup	app/main.py: lifespan(), FastAPI(...), SessionMiddleware	Source inspection; web workflow tests	Implemented and partially tested
Password hashing	app/security.py: hash_password(), verify_password()	Source inspection	Implemented; standard test suite passes
CSRF and role helpers	app/security.py: ensure_csrf(), verify_csrf(), require_roles(); form route checks in app/main.py	Source inspection; web workflow tests	Implemented and tested for selected workflows
SQLite schema/forward additions	app/db.py: SCHEMA, init_db(), _backup_before_vendor_migration()	Source inspection; tests/test_db_migrations.py	Implemented and tested for selected migrations
File validation and storage	app/main.py: check_upload_bytes(), store_bytes()	Source inspection; upload workflow tests	Implemented and tested for selected formats/cases
Native text extraction/OCR	app/services/extraction.py: _has_usable_pdf_text(), _ocr_pil_image(), extract_document()	Source inspection; tests/test_extraction_risk.py	Implemented; OCR needs local Tesseract/language resources
Structured document parser	app/services/extraction.py: extract_fields() and parser helpers	Source inspection; extraction/revision tests	Implemented; unseen layouts remain a limitation
Currency and amount parsing	app/currency.py: detect_currency(), parse_money_amount(), format_currency()	Source inspection; tests/test_currency.py, tests/test_revision_features.py	Implemented and tested for repository fixtures
Optional provider integration	app/services/llm_extraction.py: _provider_for(), optional_llm_extract()	Source inspection; mocked tests/test_llm_traceability.py	Implemented; live provider not verified

Documentation claim	Source reference	Verification method	Status
Invoice processing/status	app/services/processing.py: process_invoice(), record_extraction_trace()	Source inspection; revision/web tests	Implemented and tested for selected flows
Vendor tax/name normalization	app/services/ vendor_resolution.py: normalize_tax_id(), normalize_legal_name(), classify_po_vendor_identity()	Source inspection; tests/test_vendor_resolution.py	Implemented and tested
Authorized PO/vendor link	app/services/ vendor_resolution.py: link_po_to_vendor(); app/main.py PO link route	Source inspection; authorization/vendor-resolution tests	Implemented and tested for specified cases
PO relinking and idempotency	app/services/ vendor_resolution.py: relink_unlinked_purchase_orders(); vendor approval route in app/main.py	Source inspection; tests/test_vendor_resolution.py	Implemented and tested for fixtures
Risk assessment and formulas	app/services/risk.py: evaluate_invoice()	Source inspection; tests/test_extraction_risk.py, tests/test_revision_features.py, vendor-resolution tests	Implemented and tested for selected rules; thresholds are demo policy
Approval and simulated payment	app/main.py: /invoices/{invoice_id}/approve, / approvals/{request_id}/decision	Source inspection; tests/test_web_workflows.py	Implemented and tested for selected cases; no real payment integration
Audit events	app/services/audit.py: audit() and calls from routes/services	Source inspection; traceability and workflow tests	Implemented; local log is not immutable
Excel export	app/services/exporter.py: build_audit_workbook()	Source inspection; test_excel_export_contains_three_real_data_sheets	Implemented and tested
Latest archive contains separate security assessment file	README references it; file absent from Sentrion_MVP(6).zip	Archive file listing	Not present; add as a separate submission artifact
Dataset-wide accuracy claim	Dataset v4 not included in this archive	Archive inspection	Not verified / not performed for this deliverable
Live Gemini behavior	Tests are mocked; no provider credentials/request used	Test configuration inspection	Not verified
Hosted deployment	README says no verified hosted deployment included in this archive	README/source deployment configuration	Not verified

Appendix B. Verification Record

Source: Sentrion_MVP(6).zip, extracted to a temporary working directory without modifying the archive.

Application version: 0.2.6 (app/config.py).

Source archive SHA-256: 52318d8fda664f6d2c447cf8c541553afa748c05205468fe4b09ea9cfac89cdb.

Commands executed:

```
python -m pytest -q
# Result: 71 passed in 9.15s

python -m compileall -q app tools_*.py tests
# Result: exit code 0 (COMPILEALL_PASS)
```

Not performed: live external LLM requests, 100-pair Dataset v4 benchmark, Vercel/hosted deployment smoke test, penetration test, external bank verification, or formal license audit.